SIMATS ENGINEERING

ASSESSMENT TOOL 1 – COMPLETE ANSWER

Course: Artificial Intelligence | Course Code: CSA17

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and models for classification and decision-making.

Student Name: _____ **Reg. No:** _____

Duration: 45 Minutes **Total Marks:** 20

Note: The programs below are complete, executable Python solutions. Expected numerical output can vary slightly depending on the train/test split and random seed, so the code prints the exact values produced during execution.

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and scikit-learn.

1(a) Load, preprocess, inspect and split the dataset

```
# EXPERIMENT 1(a)
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))

# Check missing values
print("Missing values:\n", df.isnull().sum())

# Encoding target labels (demonstration)
le = LabelEncoder()
df["encoded_species"] = le.fit_transform(df["species"])

# Display first 5 rows and data types
print("\nFirst 5 rows:")
print(df.head())

print("\nData types:")
print(df.dtypes)

# Features and target
X = df[iris.feature_names]
y = df["encoded_species"]

# 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples :", len(X_test))
```

Explanation: The Iris dataset has 150 samples and four numerical features. There are no missing values. Since the input features are already numerical, no feature encoding is required. The target species labels are encoded as 0, 1 and 2. Stratified splitting preserves the class distribution in training and testing data.

1(b) Build the Decision Tree using Gini Index, print the tree and identify the root

```
# EXPERIMENT 1(b)
from sklearn.tree import export_text

model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
model.fit(X_train, y_train)

# Print tree structure
tree_text = export_text(
```

```
    model,
    feature_names=list(iris.feature_names)
)
print("\nDecision Tree Structure:")
print(tree_text)

# Root node
root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]

print("\nRoot feature:", iris.feature_names[root_feature_index])
print("Root threshold:", root_threshold)

# Feature importance
importance = pd.Series(
    model.feature_importances_,
    index=iris.feature_names
).sort_values(ascending=False)

print("\nFeature importance:")
print(importance)
```

Root-node justification: With the stated random seed, the tree normally selects **petal length (cm)** as the root feature (the exact threshold is printed by the program). The Gini criterion selects the split that gives the largest reduction in class impurity. Thus the root is the feature/split that most effectively separates the Iris classes at the first decision.

1(c) Accuracy, confusion matrix, classification report and misclassified instances

```
# EXPERIMENT 1(c)
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=iris.target_names
))

# List misclassified instances
test_result = X_test.copy()
test_result["Actual"] = y_test
test_result["Predicted"] = y_pred
misclassified = test_result[test_result["Actual"] != test_result["Predicted"]]

print("\nMisclassified instances:")
print(misclassified)

# Convert numeric labels to names
for idx, row in misclassified.iterrows():
    actual_name = iris.target_names[int(row["Actual"])]
    predicted_name = iris.target_names[int(row["Predicted"])]
    print(
        "Index:", idx,
        "| Actual:", actual_name,
        "| Predicted:", predicted_name,
        "| Features:", list(row[iris.feature_names].round(2))
    )
```

Expected performance/comment: The Iris decision tree generally gives high test accuracy, commonly around 90–100% for this split. The confusion matrix shows the number of correct and incorrect predictions for each class. Setosa is usually classified almost perfectly, while the small number of errors normally occurs between Versicolor and Virginica because their measurements overlap.

Misclassified instances: The code explicitly prints every misclassified test sample, including its index, actual class, predicted class and four feature values. Therefore, the required “at least two misclassified instances” can be reported directly from the execution output. If this particular split produces fewer than two errors, report all errors honestly rather than inventing instances.

Conclusion – Experiment 1: The Decision Tree successfully learned class boundaries from the Iris dataset. Gini impurity selected informative splits, and the high accuracy and classification metrics demonstrate that the model performs well for this small classification problem.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective: Implement Q-Learning for a 4×4 Grid World and obtain the agent's learned optimal policy.

2(a) Environment, states, actions, rewards and hyperparameters

States: 16 grid cells numbered 0–15 in row-major order. State 15 is the goal. State 5 is an obstacle.

Actions: 0 = Up, 1 = Down, 2 = Left, 3 = Right.

Rewards: +10 for reaching the goal, –1 for every normal step, –5 for attempting to enter the obstacle. An obstacle attempt keeps the agent in the current state. Reaching the goal ends the episode.

Hyperparameters: α (learning rate) = 0.8, γ (discount factor) = 0.95, ε (exploration rate) = 0.20. Episodes = 100.

```
# EXPERIMENT 2
import numpy as np
import matplotlib.pyplot as plt

# 4 x 4 Grid World
ROWS, COLS = 4, 4
N_STATES = ROWS * COLS
ACTIONS = ["U", "D", "L", "R"]
N_ACTIONS = 4

START = 0
GOAL = 15
OBSTACLE = 5

alpha = 0.8
gamma = 0.95
epsilon = 0.20
episodes = 100

Q = np.zeros((N_STATES, N_ACTIONS))
rng = np.random.default_rng(42)

def step(state, action):
    r, c = divmod(state, COLS)

    nr, nc = r, c
    if action == 0:       # Up
        nr -= 1
    elif action == 1:     # Down
        nr += 1
    elif action == 2:     # Left
        nc -= 1
    else:                 # Right
        nc += 1

    # Boundary: remain in the same state
    if nr < 0 or nr >= ROWS or nc < 0 or nc >= COLS:
        return state, -1, False

    next_state = nr * COLS + nc

    # Obstacle
    if next_state == OBSTACLE:
        return state, -5, False

    # Goal
    if next_state == GOAL:
        return next_state, 10, True

    # Normal move
    return next_state, -1, False

def choose_action(state):
    if rng.random() < epsilon:
        return int(rng.integers(N_ACTIONS))
    return int(np.argmax(Q[state]))

representative_states = [0, 10]
snapshots = {}

rewards_per_episode = []

for ep in range(1, episodes + 1):
    state = START
    total_reward = 0

    for _ in range(100):  # safety limit for each episode
        action = choose_action(state)
        next_state, reward, done = step(state, action)

        # Q-learning update
```

```
        best_next = np.max(Q[next_state])
        Q[state, action] += alpha * (
            reward + gamma * best_next - Q[state, action]
        )

        total_reward += reward
        state = next_state

        if done:
            break

    rewards_per_episode.append(total_reward)

    if ep in [1, 50, 100]:
        snapshots[ep] = Q[representative_states].copy()

print("Q-values for states", representative_states)
for ep in [1, 50, 100]:
    print(f"\nEpisode {ep}")
    for i, state in enumerate(representative_states):
        print(f"State {state} [U,D,L,R]:",
              np.round(snapshots[ep][i], 3))

# Final policy
policy = []
for s in range(N_STATES):
    if s == GOAL:
        policy.append("G")
    elif s == OBSTACLE:
        policy.append("X")
    else:
        policy.append(ACTIONS[np.argmax(Q[s])])

print("\nFinal Learned Policy:")
for r in range(ROWS):
    print(policy[r*COLS:(r+1)*COLS])

# Plot cumulative reward per episode
cumulative_rewards = np.cumsum(rewards_per_episode)

plt.figure(figsize=(8, 4.5))
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)
plt.tight_layout()
plt.show()

# Optional: show final Q-table
print("\nFinal Q-table:")
print(np.round(Q, 3))
```

2(b) Q-table evolution at Episodes 1, 50 and 100

The program records Q-values for representative states 0 and 10 at Episodes 1, 50 and 100. At Episode 1, most Q-values remain near zero because the agent has little experience. By Episode 50, actions that lead toward the goal generally acquire larger values. By Episode 100, the values become more stable and the best action in each state represents the learned policy. Because Q-Learning is stochastic, the exact printed values depend on the random seed and the environment definition; the supplied code fixes the seed for reproducibility.

2(c) Final policy, cumulative reward and convergence justification

Policy representation: U = Up, D = Down, L = Left, R = Right, X = obstacle, G = goal. The program prints the 4×4 policy grid after training.

Expected optimal path from State 0: A shortest safe route is generally $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7 \rightarrow 11 \rightarrow 15$, corresponding to **R** → **R** → **R** → **D** → **D** → **D**. Other equally good paths may be learned depending on the obstacle/reward dynamics and tie-breaking.

Convergence: The cumulative-reward curve should become increasingly stable as the agent learns which actions avoid the obstacle and reach the goal. Early episodes contain more exploration and therefore lower rewards. Later episodes exploit high-valued actions, producing more successful trajectories. Because ε remains at 0.20 throughout training, some exploration continues, so the curve need not become perfectly smooth.

Conclusion – Experiment 2: Q-Learning successfully learns action values without requiring a predefined model of the environment. Repeated interaction updates the Q-table, allowing the agent to discover a high-reward path from the start state to the goal while avoiding the obstacle.

SHORT RESULTS / OBSERVATION TO WRITE IN THE RECORD

Experiment	Result / Observation
Decision Tree	Iris classes were classified with high accuracy. Gini impurity selected the most informative root split. Confusion matrix and
Q-Learning	Q-values increased for useful actions over episodes. The final policy directed the agent toward the goal while avoiding the

Important: For the lab record, run the code once and copy the actual accuracy, confusion matrix, classification report, misclassified rows, Episode 1/50/100 Q-values, final policy and cumulative-reward plot into the Results section.